ARCSWEEP

Dust-to-USDC Aggregator for the Arc Ecosystem

Technical Whitepaper | Version 1.0

May 2026

Mike Osaro

Abstract

ArcSweep is an open-source, browser-native utility built on Circle's Arc Testnet that solves a pervasive yet largely ignored UX problem in decentralised finance: the accumulation of low-value token residues commonly called dust across wallet addresses. These micro-balances are too small to swap individually on a standard DEX without losing value to gas fees and slippage, yet they aggregate into meaningful economic waste at scale.

ArcSweep addresses this by providing a single-page application that scans an Arc Testnet wallet for sub-threshold token balances, routes each token through a pluggable multi-adapter DEX aggregation engine, selects the highest-yielding swap path per token, and batches the resulting approvals and swap calldata into a single reviewable sweep transaction outputting consolidated USDC. Gas for all transactions is denominated in USDC, consistent with Arc's stablecoin-native fee model.

This whitepaper documents ArcSweep's current architecture, its adapter protocol, the user interaction model and safety design, and provides a detailed technical roadmap including a proposed cross-chain Bridge Sweep module that extends dust reclamation to tokens stranded on foreign chains.

Table of Contents

1. Introduction & Motivation

Active participation in DeFi protocols yield farming, liquidity provisioning, airdrop farming, testnet incentive campaigns invariably generates residual token balances. Each swap, reward claim, or protocol interaction leaves behind fractional amounts that, individually, carry less economic value than the gas required to move them. Over time these remnants accumulate into what is collectively termed wallet dust.

The problem is compounded on new or emerging chains where token liquidity is still maturing. On Circle's Arc Testnet an EVM-compatible Layer 1 built around USDC-as-gas and stablecoin-native settlement early builders and ecosystem participants are particularly likely to accumulate dust from DEX experiments, incentive distributions, and smart-contract deployments.

ArcSweep was created to eliminate this friction. By combining on-chain dust detection, multi-DEX quote aggregation, and a safe one-click sweep interface, ArcSweep turns idle micro-balances back into spendable USDC the native unit of account on Arc.

Key Value Proposition

ArcSweep recovers economic value that would otherwise be permanently stranded, while serving as a reference implementation for Arc Testnet DEX integration patterns, adapter design, and USDC-denominated transaction flows.

2. Background: Arc Testnet & the Dust Problem

2.1 The Arc Blockchain

Arc is Circle's open EVM-compatible Layer 1, built on Malachite BFT consensus and designed as an Economic OS for the internet. Its distinguishing characteristics are:

- USDC as Gas transaction fees are paid in USDC, eliminating native-token volatility risk.
- Deterministic sub-second finality Malachite consensus provides near-instant settlement.
- Circle Platform Integration native access to USDC, CCTP cross-chain transfers, and Circle Wallets.
- Stablecoin-native infrastructure purpose-built for payments, FX, treasury management, and financial applications.

Arc Testnet entered public availability in April 2026, with active developer adoption ahead of the mainnet launch. Its EVM compatibility means existing Ethereum tooling, wallets (MetaMask, WalletConnect), and token standards work out of the box.

2.2 What is Wallet Dust?

In the context of EVM-compatible blockchains, dust refers to any token balance whose USD-equivalent value falls below a threshold at which a standard DEX swap would be economically rational typically under $2–$5, depending on prevailing gas prices. Dust accumulates through:

- Swap remainders fractional tokens left after rounding in AMM swaps.
- Airdrop distributions round-number token grants that, after price changes, become sub-threshold.
- Yield accruals per-block reward accumulation in small increments.
- Testnet incentives protocol-team distributions of experimental tokens.
- Failed transaction residues partially consumed allowances or partial fills.

2.3 Why Existing Solutions Fall Short

Approach	Limitation
Manual individual swaps	Gas cost exceeds token value; wastes time
Centralized exchange dust sweeps	Requires custody transfer; not available on Arc
Ignore the dust	Value is permanently locked; wallet clutter grows
Custom batch scripts	Requires developer skill; no safety checks; single DEX only

3. ArcSweep Architecture

ArcSweep is a self-contained single-page application (SPA) served by a lightweight Node.js HTTP server (server.mjs). It has no backend database, no server-side wallet custody, and no external API dependencies all sensitive operations execute client-side within the user's browser via their injected wallet provider (window.ethereum).

3.1 High-Level Data Flow

Connect: User connects an EVM wallet (e.g. MetaMask) to the Arc Testnet. ArcSweep reads the connected address and displays the network pill confirming Arc Testnet.

Scan: DustCleaner.detectDust() queries the token registry (tokens.js) and calls balanceOf for each known token, filtering those whose USD value falls below the configurable dust threshold (default $2.00).

Quote: DustCleaner.getBestQuotes() fans out concurrent HTTP requests to all registered DEX adapters, requesting swap quotes for each dust token. Adapters compete on outputAmount.

Select: Best quotes are ranked by net USDC output. The user may auto-select all or manually curate the token list. A live summary (detected value, best output, selected count) updates reactively.

Review: DustCleaner.buildSweep() assembles ERC-20 approve + swap calldata for each selected token. A modal review dialog presents the full transaction list before any wallet interaction.

Sweep: On confirmation, ArcSweep sequentially submits approvals and swaps to the Arc Testnet, monitoring transaction receipts and surfacing results in the UI.

4. Core Modules

4.1 DustCleaner Class (src/arcSweep.js)

The DustCleaner class is the programmatic heart of ArcSweep and is designed to be importable as a standalone library (import { DustCleaner } from 'arcsweep'). It accepts a configuration object at construction containing:

- provider — an EIP-1193-compatible provider (window.ethereum or equivalent).
- account — the wallet address to scan.
- adapters — an array of QuoteAdapter instances.

detectDust({ thresholdUsd })

Iterates the token registry, calls balanceOf(account) for each token via the provider, converts raw balances using each token's decimals, fetches USD price data, and returns tokens whose USD value is below thresholdUsd. Returns an array of DustToken objects each containing: address, symbol, decimals, rawBalance, usdValue.

getBestQuotes(dustTokens, { slippageBps })

For each dust token, fans out simultaneous HTTP requests to all registered adapters. Each adapter returns a QuoteResult or null. getBestQuotes picks the adapter with the highest outputAmount per token and attaches it to the DustToken. Adapter validation rejects: zero-address routers, empty calldata, missing output amounts, or price impact above configurable thresholds.

buildSweep(quotedTokens)

Transforms each QuotedToken into a pair of on-chain calls ERC-20 approve (router, amount) and the router swap call encoded in quote.tx. Returns a SweepPlan: an ordered list of TransactionRequest objects ready for submission via eth_sendTransaction.

4.2 Token Registry (src/tokens.js)

A static registry of Arc Testnet token contracts, each entry specifying address, symbol, decimals, and an optional coingeckoId for price resolution. The registry is intentionally kept minimal and chain-specific to avoid false positives from cross-chain token address collisions. Operators may extend it by modifying the array before deployment.

4.3 Adapter System (src/adapters/mockArcDexes.js)

The adapter layer is a pluggable abstraction over DEX HTTP APIs. The default export reads live configuration from window.ARCSWEEP_DEXES or localStorage['arcsweep:dexes'], enabling deployment-time customisation without source changes. The factory function createHttpQuoteAdapter accepts:

Field	Description
id	Unique string identifier for the DEX
name	Human-readable name displayed in the UI route panel
quoteUrl	HTTP endpoint implementing the ArcSweep Quote API
routerAddress	Verified Arc Testnet router contract address

5. Quote Adapter Protocol

Any DEX wishing to integrate with ArcSweep must expose an HTTP endpoint conforming to the ArcSweep Quote API. This is intentionally simple to minimise integration effort.

5.1 Request Parameters

Parameter	Type	Description
sellToken	address	ERC-20 token contract to sell
sellAmount	uint256 string	Raw token amount (pre-decimals)
buyToken	address	Target token (USDC on Arc Testnet)
taker	address	Wallet address initiating the swap
slippageBps	integer	Maximum acceptable slippage in basis points
chainId	integer	Arc Testnet chain ID

5.2 Response Schema

Field	Type	Description
outputAmount	string	Expected USDC received (human-readable)
minimumOutputAmount	string	Slippage-adjusted minimum output
priceImpactBps	integer	Estimated price impact in basis points
route	string[]	Token path (e.g. [TOKEN, WETH, USDC])
tx.to	address	Router contract to call
tx.data	hex string	Encoded swap calldata
tx.value	hex string	Native value to send (usually 0x0)

5.3 Adapter Validation Rules

The adapter factory enforces the following hard rules before accepting a quote:

- routerAddress must not be the zero address.
- tx.data must be non-empty.
- outputAmount must be a positive finite number.
- priceImpactBps must not exceed the configured maximum (default 300 bps / 3%).
- route array must contain at least two tokens (sell → buy).

6. User Interface & Safety Design

6.1 Layout

The UI is structured in a three-panel workspace: a Scanner panel (left) housing scan controls, a Detected Dust panel (centre) with a sortable token table, and a Best Quote panel (right) showing per-token route details. A top bar provides wallet connection, network status, and branding.

A summary quote board above the workspace displays live totals: Detected value (sum of all dust USD values), Best output (expected USDC after all swaps), and Selected (token count).

6.2 Scanner Controls

Control	Description
Dust limit slider ($0.01–$10.00)	Sets the USD threshold below which a token balance is classified as dust
Slippage selector (0.25% / 0.5% / 1%)	Passed as slippageBps to all adapter quote requests
Priority selector	Best return: maximises USDC output. Lowest hops: minimises multi-hop routes
Auto-select checkbox	Automatically selects all detected dust tokens after a scan
Scan wallet button	Triggers detectDust + getBestQuotes pipeline
Review sweep button	Opens the pre-signing review dialog (disabled until at least one token is selected)

6.3 Review Dialog & Signing Safety

Before any wallet interaction occurs, ArcSweep presents a full Review Sweep modal listing every pending transaction: token symbol, sell amount, expected USDC output, router address, and price impact. The user must explicitly confirm in the dialog before the first eth_sendTransaction call is made.

> **Security Guarantee**
>
> ArcSweep never requests a seed phrase, private key, or raw signature. All signing is delegated entirely to the user's connected wallet. The application holds no custody of funds at any point.

7. Security Considerations

7.1 Current Mitigations

- Path traversal protection: server.mjs validates that every resolved file path is a subdirectory of the project root before serving, returning HTTP 403 on directory escape attempts.
- Router address validation: adapters reject the zero address and any address not matching a verified format.
- Calldata validation: empty or malformed tx.data fields are rejected before buildSweep.
- No seed phrase exposure: the application never prompts for or stores private key material.
- No server-side state: zero attack surface from a compromised backend.

- Cache-control: no-store header on all served assets prevents stale code execution.
- Review gate: the mandatory pre-signing dialog prevents accidental or malicious calldata from being signed without explicit user review.

7.2 Recommended Hardening for Production Use

- Serve ArcSweep over HTTPS (TLS) to prevent man-in-the-middle injection of malicious adapter configs.
- Pin the ARCSWEEP_DEXES config to an integrity-checked JSON endpoint rather than inline script injection.
- Add a Subresource Integrity (SRI) check on the app.js module if hosting on a CDN.
- Consider a Content Security Policy (CSP) header restricting script sources to self.
- For mainnet deployment, introduce a smart-contract-based router allowlist to prevent adapter spoofing.

8. Future Roadmap: Cross-Chain Bridge Sweep

The single most significant expansion opportunity for ArcSweep is the addition of a Bridge Sweep module a cross-chain dust reclamation pipeline that extends the platform's reach beyond Arc Testnet to tokens stranded on other EVM (and non-EVM) networks. This section documents the proposed architecture in detail.

8.1 The Cross-Chain Dust Problem

A user active across multiple chains Ethereum mainnet, Arbitrum, Optimism, Base, Polygon, BNB Smart Chain will inevitably accumulate dust on each network independently. These tokens cannot be swept locally to USDC on Arc because they exist on separate execution environments with separate liquidity pools. Moving them individually incurs bridging fees that frequently exceed the token's value.

Bridge Sweep solves this by aggregating cross-chain dust into a single coordinated operation: swap each token to a bridge-compatible stablecoin on its native chain, bridge the resulting stablecoin to Arc via Circle's Cross-Chain Transfer Protocol (CCTP), and consolidate all inbound transfers into a single USDC balance on Arc.

8.2 Proposed Architecture

Phase 1 — Multi-Chain Dust Detection

Extend DustCleaner.detectDust() to accept a chains configuration array. For each chain, ArcSweep would:

- Connect to a read-only RPC endpoint for the target chain.
- Query the chain-specific token registry for the user's address.
- Fetch USD prices and apply the dust threshold.
- Return a cross-chain DustManifest grouping dust tokens by their origin chain.

Phase 2 — Source-Chain Swap to Bridge Asset

On each origin chain, ArcSweep would route dust tokens through that chain's DEX ecosystem using the same adapter pattern already in place for Arc. The target buy token on each chain would be a CCTP-compatible asset USDC on CCTP-supported chains, or a highly liquid stablecoin (USDT, DAI) on others, subsequently swapped to USDC before bridging.

This phase requires chain-specific adapter registries the same pluggable createHttpQuoteAdapter factory can be reused with chain-aware endpoint configurations. The critical addition is a source-chain gas estimator that vetoes sweeps where expected output minus bridge fee minus gas cost is negative.

Phase 3 — CCTP Bridge Execution

Circle's Cross-Chain Transfer Protocol (CCTP) is the natural bridge choice given ArcSweep's Arc/USDC-centric design. CCTP burns USDC on the source chain and mints an equivalent amount on Arc, with Circle attestation providing the cryptographic proof of burn.

The Bridge Sweep module would orchestrate the CCTP flow:

Step 1: approve (TokenMessenger, usdcAmount) grant CCTP permission to burn the swapped USDC.

Step 2: depositForBurn (amount, arcDomain, recipient, burnToken) initiate the burn on the source chain.

Step 3: Poll Circle's Attestation Service API for the signed attestation message.

Step 4: receiveMessage (message, attestation) mint USDC on Arc Testnet at the user's address.

Phase 4 — Non-CCTP Chain Support

For chains not yet supported by CCTP, ArcSweep would integrate third-party bridge aggregators (e.g. Li.Fi, Socket, Stargate) via an extensible BridgeAdapter interface mirroring the existing QuoteAdapter pattern. Each BridgeAdapter would expose:

Method	Returns	Description
getBridgeQuote(asset, amount, srcChain, dstChain)	BridgeQuote	Fee, estimated time, route
buildBridgeTx(quote, recipient)	TransactionRequest[]	Executable bridge calldata
pollStatus(txHash, srcChain)	BridgeStatus	Pending / completed / failed

Phase 5 — Unified Cross-Chain Review UI

The existing Review Sweep dialog would be extended into a multi-chain sweep summary, grouping transactions by chain and presenting:

- Per-chain dust total (USD value of tokens being swept on that chain).
- Per-chain gas cost estimate (denominated in the chain's native token and USD equivalent).
- Bridge fee and estimated transit time for each CCTP or third-party bridge hop.
- Expected net USDC landing on Arc after all fees.

- A chain-by-chain confirmation flow, allowing users to cancel individual chain sweeps without aborting the entire operation.

8.3 Economic Model for Bridge Sweep

Cross-chain sweeps introduce multiple fee layers that must all be surfaced to the user before signing. The Bridge Sweep module will enforce a minimum net value threshold: if expected USDC output minus all fees (source gas + bridge fee + Arc gas) is less than a configurable floor (proposed: $1.00), the sweep is disabled and a clear explanation is shown.

Fee Component	Paid In	Estimated Range
Source chain swap gas	Source chain native token	$0.01 – $5.00 (chain-dependent)
CCTP burn transaction gas	Source chain native token	$0.50 – $3.00
CCTP attestation wait	Time cost (no fee)	~15 seconds on supported chains
Arc Testnet receive gas	USDC (Arc native)	Sub-cent (Arc's gas model)
Bridge Sweep protocol fee (proposed)	USDC deducted on Arc	0.1% of swept amount

8.4 Supported Chain Roadmap

Chain	Bridge Protocol	Priority	Status
Ethereum Mainnet	CCTP v2	P0	Planned
Arbitrum One	CCTP v2	P0	Planned
Base	CCTP v2	P0	Planned
Optimism	CCTP v2	P1	Planned
Polygon PoS	CCTP v2	P1	Planned
BNB Smart Chain	Li.Fi / Socket	P2	Research
Avalanche C-Chain	CCTP v2	P2	Planned
Solana	CCTP v2 (non-EVM)	P3	Research

8.5 Smart Contract Requirements

Bridge Sweep's on-chain components will require audited contracts for:

- SweepBatcher — a router contract on Arc that receives multiple CCTP mints in a single session and batches any residual on-Arc swaps into a final USDC consolidation, reducing the number of Arc Testnet transactions to one per sweep session.
- CrossChainDustRegistry — an optional on-chain registry mapping (chainId, tokenAddress) to a canonical Arc Testnet equivalent, enabling automatic route resolution without manual configuration.
- FeeCollector — a minimal upgradeable contract managing protocol sweep fees with a DAO-controlled fee parameter.

9. Tokenomics & Fee Model (Proposed)

ArcSweep v0.1.0 is feeless. The following tokenomics model is proposed for a mainnet production release as a means of sustaining protocol development and incentivising liquidity and adapter contributions.

9.1 Protocol Fee

A protocol fee of 0.1% of swept USDC output, collected by the FeeCollector contract on Arc. At this rate, a user sweeping $50 of dust pays $0.05, negligible relative to recovered value. The fee is deducted from the USDC output after the swap, not charged additionally.

9.2 Fee Distribution

Recipient	Share	Purpose
Protocol treasury	50%	Development, audits, infrastructure
Adapter operators	30%	Proportional to quote volume served
Liquidity incentives	20%	Rewards for future liquidity event

9.3 SWEEP Governance Token (Proposed)

A governance token (ticker: SWEEP) is proposed for decentralised control of protocol parameters: dust threshold defaults, fee rates, adapter allowlisting, bridge protocol whitelisting, and treasury allocation. SWEEP holders would vote via a lightweight on-Arc governance contract. No public sale or VC allocation is envisioned; initial distribution would reward:

- Testnet and Mainnet participants — addresses that used ArcSweep on Arc Testnet and Mainnet.
- Adapter contributors — teams that shipped and maintained verified DEX adapters.

- Open-source contributors — merged pull requests to the core repository.

10. Governance & Open-Source Licensing

ArcSweep is published under the MIT License, permitting unrestricted use, modification, and commercial deployment. The repository is hosted publicly at github.

In the current testnet phase, protocol decisions (token registry updates, adapter approvals, version releases) are made by the core contributor team. The proposed SWEEP governance model would transition these decisions on-chain as the protocol matures post mainnet.

ArcSweep explicitly does not seek to monopolise the Arc ecosystem's DEX aggregation layer. The adapter protocol is designed to be implemented by any DEX team with minimal effort, and the DustCleaner library is intended to be embedded in third-party wallets, dashboards, and portfolio tools.

11. Conclusion

ArcSweep addresses a real and underserved problem in the Arc ecosystem: the silent erosion of user value through stranded dust balances. By combining a clean, safety-first user interface with a flexible multi-adapter aggregation engine, ArcSweep turns economic waste into usable USDC with a single reviewed click.

The protocol is deliberately minimal today — a single-chain, testnet-native tool built without external dependencies. This simplicity is a strength: it keeps the attack surface small, makes the codebase auditable, and ensures the core value proposition is proven before adding complexity.

The Bridge Sweep roadmap represents the protocol's natural evolution: as Arc approaches mainnet and its ecosystem deepens, users will increasingly hold cross-chain balances. Bridge Sweep, built on CCTP and a permissive bridge adapter interface, positions ArcSweep as the definitive dust reclamation layer for the Arc stablecoin economy.

Get Involved

Reach out to core contributor on : github.com/mosaro0224/ Email: ogiemichael.om@gmail.com
Contribute a DEX adapter, extend the token registry, or submit a pull request.
ArcSweep welcomes all contributions.